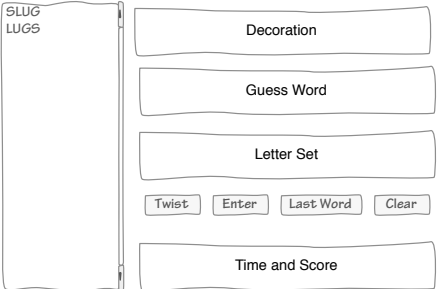


In this programming lab project, we are going to develop a version of text twist game in which we present a random assortment of letters to the operator/player. We expect the operator to derive as many words as possible given the current list of letters. We award points to the operator for each successful word based on the number of characters in that word.



Layout template for your GUI. The image on the right is inspirational only. You are expected to use user interface components (like buttons and fields, and labels) with some image decoration.



The little boxes to the left are not appropriate for us. We will use that area as an active list of valid words guessed by the operator

In the human interface, we see the following:

- a scrollable list view of used (successful guess) words, not to be used/allowed again.
- a single Text or a collection of up to 10 Text items whose text displays the sequence of letters in the order selected by the user (the user's current guess word).
- a collection of buttons (one per letter in the current letter set). When the user pushes a button, they are selecting that letter on that button at that time to be appended to their current guess word
- a pane for displaying the overall game score and the score for the current level, and time left on the current level, etc.
- buttons to manipulate the guess word and letter set: btnTwist, btnEnter, btnLast, btnClear

- 1) Create a new MAVEN project for this lab. **Use the class naming convention for the maven coordinates.** In the new Maven Project dialog...
 - Use the openjfx archetype fxm1
 - Group id: cs120
 - Artifact id: lab06
 - uncheck the "run archetype generator interactively" checkbox
 - javafx-version: 21
- 2) From within your package explorer view, select the src folder and create the missing source folders:
 - src/test/java
 - src/test/resources
- 3) Edit the pom.xml file and add (copy/cut/paste) the following inside the <dependencies> tag container:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
```

- 4) Copy/decompress dictionary files and add them into your top level project folder. Do NOT add the files to your src directory. The dictionary file should be residing in your top-level project folder.

-
- 5) Create **WordDictionary** class to define a helper object whose job is to lookup/check for correctly spelled words, and select a random word of the correct length (size in characters -- n). What methods do you need for this kind of object? See the UML diagram below and Appendix A.
 - 6) **Testing:** Use JUnit testing to make sure your **WordDictionary** class is working.
 - 7) **Scoring:** 3 letter words award 3*10 points per letter, 4 letter words awards 4*10 points per letter, ... n letter words awards n*10 points per letter.
 - 8) **GUI:** Create a new MVC JavaFX human interface with impeccable style (controlled by external CSS file).
 - 9) **Buttons:**
 - The **twist** button, clears the *guess-word* and reshuffles the *letter set*. Shuffling can be achieved by repeated swapping of randomly selected letters.
 - The **enter** button causes the game to check the *guess-word* and award points if necessary. If the user guesses the max word (involving all letters), the game is won. Advance to next level.
 - The **last word** button is a short cut for retyping the same letters as the last guess word in the same order. This allows a user to quickly add a suffix.
 - The **clear** button clears the current *guess word* and returns all the letters to the *letter set*.
 - Letters in the letter (letter buttons) may only be used once. Disable the letter buttons as they are used. Re-enable them when the users starts a new guess.
 - 10) Use a timer to decrement a count-down counter (once a second). At the beginning of an episode (with a given *letter set*, the user is limited to 2 minutes (120 seconds). Show the time left in minutes:seconds format.

At the end of the episode, the game determines if the user has guessed enough to allow the user to:

 - proceed at the same level with a new letter set ... OR ...
 - advance to the next level with a new letter set, with one more letter... OR...
 - terminate the game.
 - 11) For an episode (a given letter set) pick a random word from the dictionary of suitable size dictated by the current level. Level 1 starts with a 3 letter word set. Level n requires n+2 letters in letter set. Create and use your Dictionary helper object (configured for the proper number of letters). Use this business object until you advance to the next letter. Replace with a newly configured instance of a Dictionary.
 - 12) For an episode, the user must acquire $\text{floor}(0.25 * \text{max}^2 * 10)$ points in order to stay alive and continue on the current level. If the user fails to do so, the game is over. For example, on level 5, the user is dealing with 7 letter sets. The max word(s) is max=7 letters long. The minimum threshold to stay alive is $\text{floor}(0.25 * 7 * 7 * 10) = \text{floor}(0.25 * 490) = \text{floor}(122.5) = 122$. So the user would have to get at least some combination of words adding up beyond 122 in order to stay alive and continue at this level.
 - 13) When the user guesses the max long word for an episode (using all letters), the episode terminates and the user advances to the next level (adding one more letter in the letter set).
 - 14) The game is “won” when the user finishes level 8 (10 letter words).

Appendix A:

Class Name:	WordDictionary	
Package:		
Extends:		
Instance variables:	folderName: String other variables of your choice	relative path to folder containing the word files in your project. appropriate variables to help you manage the collection of words
Constructors:	WordDictionary(path: String)	initializes the instance variable indicating where the data files are located in your project.
Instance methods:	<<getters and setters>>	Appropriate accessor and mutator methods, if any.
	initialize()	Opens a FileReader stream on the correct file and prepares a BufferedReader on the FileReader stream. Then reads the number of words (first line) and then reads each subsequent word from the dictionary one line at a time. Building a helper collection to make word lookups efficient.
	isValidWord(wrd:String): boolean	returns boolean; Looks up the word in the helper collection. Compares each word to the target word. If a match is found, returns true . Returns false if specified word is not found in the file. Closes the file. Returns false if IOException or FileNotFoundException occurs.
	String randomWord(n:int) : String	returns null or a string. From the helper collection, determines the number of words and selects a random integer between 0 and numWords. Randomly returns one of the words from the collection.

Appendix B:

Class Name:	TwistController	
Package:		
Extends:		
Instance variables:	score: int	current total game score
	levelScore: int	current level/episode score
	level: int	current game level; is used to determine the number of letters we use.
	targetWord: String	randomly selected target word for this episode determines the set of letters to use
	myDictionary: WordDictionary	helper object
Constructors:	TwistController()	initializes the instance variables
Instance methods:	<<getters and setters>>	Appropriate accessor and mutator methods, if any.
	shuffleLetters(): List<String>	returns a list of shuffled letter strings made from the current level's target word
	checkGuessWord(wrd: String): int	Uses the dictionary helper to see if the supplied wrd is valid. if so, then returns the appropriate points for a successful word. Returns 0 when the word is not valid.
		other call back methods you need.